HW6 for CS 6210

You may (and probably should) talk about problems with the each other, with the TA, and with me, providing attribution for any good ideas you might get. Your final write-up should be your own.

```
1 using LinearAlgebra
```

```
1 using Plots
```

1. Damped oscillation

Consider the *quadratic eigenvalue problem* of finding $\lambda \in \mathbb{C}$ and nonzero $u \in \mathbb{C}^n$ such that

$$(\lambda^2 I + \lambda \epsilon B + K)u = 0.$$

Here we assume that K (the stiffness matrix) is spd and B (the damping matrix) is symmetric and positive semidefinite. Often one writes $\lambda = i\omega$ where ω is a frequency of vibration. The problem can be converted into a larger standard eigenvalue problem by a process called *linearization* (a terrible name, but it's what we have), with the most common version being the Frobenius (or companion) linearization:

$$\begin{bmatrix} -\epsilon B & -K \\ I & 0 \end{bmatrix} \begin{bmatrix} v \\ u \end{bmatrix} = \lambda \begin{bmatrix} v \\ u \end{bmatrix}.$$

Complete the following codes to

- Compute all solutions of the quadratic eigenvalue problem via the Frobenius linearization
- Compute the derivative with respect to ϵ for a frequency ω about $\epsilon = 0$

You should do a finite difference check to test consistency of the second code with the first code.

The Frobenius linearization transforms this into a standard eigenvalue problem of size $2n \times 2n$: $\begin{bmatrix} -\epsilon B & -K \\ I & 0 \end{bmatrix} \begin{bmatrix} v \\ u \end{bmatrix} = \lambda \begin{bmatrix} v \\ u \end{bmatrix}$, where $v = \lambda u$. This is $Az = \lambda z$ with $z = [v; u]$ and $A = \begin{bmatrix} -\epsilon B & -K \\ I & 0 \end{bmatrix}$. Construct the $2n \times 2n$ matrix A. Compute its eigenvalues λ_k and eigenvectors z_k using eigen(A). Extract $\omega_k = -i\lambda_k$ (since $\lambda = i\omega \implies \omega = \lambda/i = -i\lambda$). From each $z_k = [v_k; u_k]$, extract and normalize u_k (using Euclidean norm for consistency). Return all ωs

The quadratic eigenvalue problem (QEP) is $Q(\lambda)u = 0$ where $Q(\lambda) = \lambda^2 I + \epsilon\lambda B + K$. At $\epsilon = 0$, it reduces to $(\lambda^2 I + K)u = 0$, or $Ku = -\lambda^2 u = \omega^2 u$ since $\lambda = i\omega$, $-\lambda^2 = \omega^2$. To find $\frac{d\omega}{d\epsilon}|_{\epsilon=0}$, use perturbation theory. Differentiate QEP w.r.t. ϵ: $\frac{d}{d\epsilon}[Q(\lambda)u] = 0$. $(2\lambda\frac{d\lambda}{d\epsilon} I + \lambda B)u + Q(\lambda)\frac{du}{d\epsilon} = 0$ at $\epsilon = 0$. Left-multiply by left eigenvector $w^H = u^H$ (self-adjoint at $\epsilon = 0$, since K spd, real symmetric: $u^H(2\lambda\frac{d\lambda}{d\epsilon}u + \lambda B u) = 0$ as $u^H Q(\lambda) = 0$.

$$2\lambda\frac{d\lambda}{d\epsilon} + \lambda u^H B u = 0 \implies \frac{d\lambda}{d\epsilon} = -\frac{1}{2}u^H B u$$

$$\frac{d\omega}{d\epsilon} = \frac{d}{d\epsilon}(-i\lambda) = -i\frac{d\lambda}{d\epsilon} = i\frac{1}{2}u^H B u$$

Assumes simple eigenvalues, normalized u.

`solve_qep`

[ωs, Us] = solve_qep(K, B, ϵ)

Compute all eigenpairs (i*ωs[k], Us[:,k]) for the quadratic eigenvalue problem

```
"""
[ωs, Us] = solve_qep(K, B, ϵ)

Compute all eigenpairs (i*ωs[k], Us[:,k]) for the quadratic eigenvalue problem
"""
function solve_qep(K, B, ϵ)
    n = size(K, 1)
    # Frobenius linearization (2n x 2n matrix)
    A = [-ϵ*B -K; I(n) zeros(n, n)]
    # Compute eigenvalues/vectors
    F = eigen(A)
    lambda_s = F.values
    Zs = F.vectors
    # ω = -i lambda (from λ = i ω)
    ωs = -im * lambda_s
    # Extract and normalize u's (lower half of z)
    #Us = similar(Zs, n, 2n)
    #for k in 1:2n
    #    u = Zs[n+1:end, k]
    #    Us[:, k] = u / norm(u)
    #end
    # Raw eigenvectors u
    Us = Zs[n+1:end, :]
    ωs, Us
end
```

deriv_freq

δω = deriv_freq(K, B, ω, u)

Compute δω = dω/dϵ at of a QEP solution (i*ω, u) at ϵ = 0.

```
"""
δω = deriv_freq(K, B, ω, u)

Compute δω = dω/dϵ at of a QEP solution (i*ω, u) at ϵ = 0.
"""
function deriv_freq(K, B, ω, u)
    g = dot(u, B * u) / dot(u, u) # u^H B u / u^H u (general for unnormalized u)

    (im / 2) * g
end
```

1.8001403503853413e-7

```
begin
    # Example matrice
    n = 2
    K = [2.0 0.5; 0.5 1.0]  # spd
    B = [1.0 0.2; 0.2 0.5]  # symmetric psd

    # At ϵ=0, compute undamped ωs, Us
    ωs0, Us0 = solve_qep(K, B, 0.0)

    # Pick one mode
    idx = argmax(real.(ωs0))
    ω0 = ωs0[idx]
    u0 = Us0[:, idx]

    # derivative
    δω_anal = deriv_freq(K, B, ω0, u0)

    # Finite difference
    Δϵ = 1e-6
    ωs_pert, Us_pert = solve_qep(K, B, Δϵ)
    # Find closest ω to ω0 (match by distance)
    dists = abs.(ωs_pert .- ω0)
    idx_pert = argmin(dists)
    ω_pert = ωs_pert[idx_pert]
    δω_fd = (ω_pert - ω0) / Δϵ

    # Check relative error
    rel_err = abs(δω_fd - δω_anal) / abs(δω_anal)
    rel_err
end
```

2. The Power (Series)

Consider an analytic function given by a power series

$$f(z) = \sum_{j=0}^{\infty} c_j z^j,$$

with radius of convergence ρ. Argue that if $|\lambda| < \rho$, then

$$f(J_\lambda) = \begin{bmatrix} f(\lambda) & f'(\lambda) \\ 0 & f(\lambda) \end{bmatrix}$$

where

$$J_\lambda = \begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}.$$

The Jordan canonical form $AV = VJ$, where J is block diagonal with Jordan blocks J_λ for eigenvalues λ. For a defective eigenvalue (algebraic multiplicity > geometric multiplicity), the Jordan block is upper triangular with λ on the diagonal and 1's on the superdiagonal. This represents a 1-dimensional invariant subspace (the eigenspace) nested within a 2-dimensional generalized eigenspace. Matrix functions like $f(A)$ are defined for analytic f via the power series, especially when the spectral radius (max $|\lambda_i|$) is within the radius of convergence ρ. Here, the spectral radius of J_λ is $|\lambda|$, and since $|\lambda| < \rho$, the series $\sum c_j J_\lambda^j$ converges.

From the power iteration, we see how matrix powers A^k are analyzed via diagonalization $A = V\Lambda V^{-1}$, leading to $A^k = V\Lambda^k V^{-1}$. For non-diagonalizable cases like Jordan blocks, we use a similar decomposition. Write $J_\lambda = \lambda I + N, \quad where \quad N = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$. Here, I is the 2×2 identity, and N is $N^2 = 0$, as higher powers vanish. This mirrors the sensitivity analysis, where small perturbations (like the off-diagonal 1) can amplify errors, but here it helps compute powers.

Since λI and N commute (as noted in similarity transforms from the perturbation, where similarity preserves eigenvalue structure), we can expand powers using the binomial theorem: $J_\lambda^k = (\lambda I + N)^k = \sum_{m=0}^{k} \binom{k}{m} \lambda^{k-m} N^m$. Because $N^2 = 0$, terms with $m \geq 2$ are zero, so $J_\lambda^k = \lambda^k I + k\lambda^{k-1} N = \lambda^k \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + k\lambda^{k-1} \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} \lambda^k & k\lambda^{k-1} \\ 0 & \lambda^k \end{bmatrix}$. This explicit form of J_λ^k is akin to the power iteration update $x^{(k)} = \frac{A^k x^{(0)}}{||A^k x^{(0)}||}$, where dominant terms emerge.

Since f is analytic with $|\lambda| < \rho$, the matrix function is $f(J_\lambda) = \sum_{j=0}^{\infty} c_j J_\lambda^j$. Substitute the expression for J_λ^j: $f(J_\lambda) = \sum_{j=0}^{\infty} c_j \begin{bmatrix} \lambda^j & j\lambda^{j-1} \\ 0 & \lambda^j \end{bmatrix} = \begin{bmatrix} \sum_{j=0}^{\infty} c_j \lambda^j & \sum_{j=1}^{\infty} c_j j \lambda^{j-1} \\ 0 & \sum_{j=0}^{\infty} c_j \lambda^j \end{bmatrix}$ (For $j = 0$, the off-diagonal term is 0, so the sum starts at $j = 1$.) The diagonal entries are $\sum c_j \lambda^j = f(\lambda)$. The superdiagonal entry is $\sum_{j=1}^{\infty} j c_j \lambda^{j-1}$, which is the derivative: $\frac{d}{d\lambda} f(\lambda) = \frac{d}{d\lambda} \left(\sum_{j=0}^{\infty} c_j \lambda^j \right) = \sum_{j=1}^{\infty} j c_j \lambda^{j-1} = f'(\lambda).$

Thus, $f(J_\lambda) = \begin{bmatrix} f(\lambda) & f'(\lambda) \\ 0 & f(\lambda) \end{bmatrix}$.

3. Spin me right round

Consider the power iteration for the matrix

$$A = \begin{bmatrix} 0.5 & -1 \\ 1 & 0 \end{bmatrix}$$

where the starting vector is

$$z^{(0)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Denote the ith iterate of power iteration as

$$z^{(k)} = \begin{bmatrix} x_k \\ y_k \end{bmatrix}$$

and plot the curve of (x_k, y_k) points in the plane for $k = 0, \ldots, 100$. Give a closed-form expression that explains this curve.

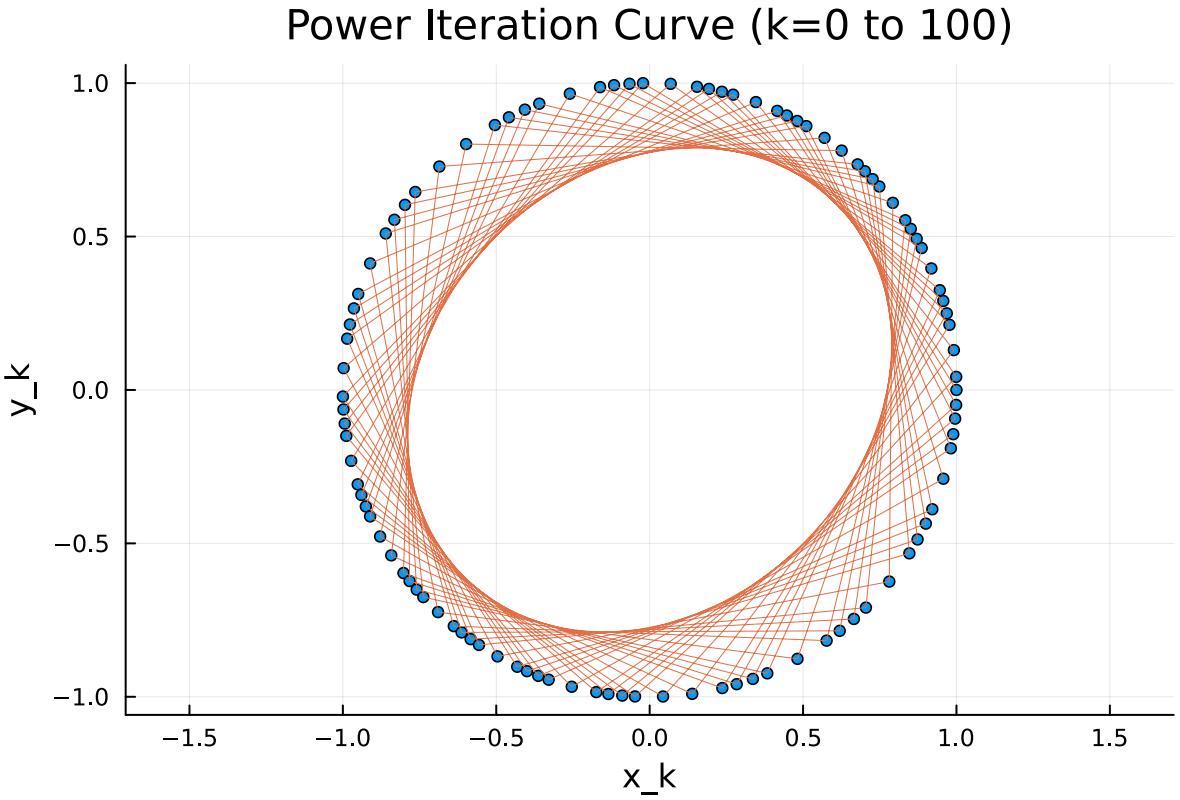
Hint: The eigenvectors are a complex conjugate pair written as $w_{\pm} = u \pm iv$, and $A \begin{bmatrix} u & v \end{bmatrix} = \begin{bmatrix} u & v \end{bmatrix} L$ where L has a simple form.

The power iteration is defined as $x^{(k+1)} = \frac{Ax^{(k)}}{||Ax^{(k)}||}$. The characteristic polynomial is $p(\lambda) = \det(A - \lambda I) = \lambda^2 - 0.5\lambda + 1 = 0$. The roots are $\lambda = \frac{0.5 \pm \sqrt{0.25 - 4}}{2} = \frac{0.5 \pm \sqrt{-3.75}}{2} = \frac{1}{4} \pm i\frac{\sqrt{15}}{4}$. These are complex conjugates with magnitude $|\lambda| = \sqrt{\left(\frac{1}{4}\right)^2 + \left(\frac{\sqrt{15}}{4}\right)^2} = \sqrt{\frac{1}{16} + \frac{15}{16}} = 1$.

Since both eigenvalues have equal magnitude 1 > 0 (no decay), power iteration does not converge to a single eigenvector. Instead, the iterates rotate due to the rotational nature of the eigenvalues.

Using spectral decomposition $A = V\Lambda V^{-1}$. $\lambda = \frac{1}{4} \pm i\frac{\sqrt{15}}{4}$. Eigenvectors V from $(A - \lambda I)v = 0$. $x^{(0)} = V\tilde{x}$. Unnormalized $z^{(k)} = A^k x^{(0)} = V(\Lambda^k \tilde{x})$. $x^{(k)} = \frac{z^{(k)}}{|z^{(k)}|}$. The curve is the unit circle $x^2 + y^2 = 1$, as normalization ensures $|x^{(k)}| = 1$. Let ϕ_0 be initial phase (from $x^{(0)}$), then $x_k = cos(\phi_0 + k\theta), y_k = sin(\phi_0 + k\theta)$, where $\theta = arg(\lambda_1)$.

Closed-Form: $(x_k, y_k) = (cos(\phi + k\theta), sin(\phi + k\theta))$, where $\theta = arg(\lambda_1)$




```
begin
    A = [0.5 -1.0; 1.0 0.0]
    z = [1.0; 0.0]
    points = zeros(2, 101)
    points[:, 1] = z
    for k in 1:100
        y = A * z
        z = y / norm(y)
        points[:, k+1] = z
    end

    # Plot: Scatter points and connect with line for the curve
    scatter(points[1, :], points[2, :], aspect_ratio=1, label="", markersize=3,
    xlabel="x_k", ylabel="y_k", title="Power Iteration Curve (k=0 to 100)")
    plot!(points[1, :], points[2, :], linewidth=0.5, label="")
end
```